

1. ¿Qué cambió en tu plan cuando separaste el trabajo en cuatro roles en vez de pensarlo "todo junto"?

Separar el desarrollo en cuatro roles automáticos (**Product Owner, Developer, Scrum Master y Architect**) transformó por completo mi planificación. Cuando uno lo piensa "todo junto", la tendencia natural es mezclar prioridades técnicas con necesidades de negocio.

Al dividirlos, el flujo cambió radicalmente:

- **Negocio vs. Estructura:** El Product Owner se enfocó estrictamente en exprimir el máximo valor del backlog, mientras que el Architect se dedicó a estructurar las decisiones de software en los archivos ADR sin interferir en la prioridad comercial.
- **Filtro de calidad inmediato:** El Scrum Master y el Developer dejaron de ser simples ejecutores para convertirse en custodios de la calidad, asegurando de forma independiente que cada historia en backlog.json estuviera lista para el Sprint antes de generar el plan final.

2. ¿Qué historia te costó más dejar lista según INVEST, y por qué?

Definitivamente me costó mucho la **US-07 (Acceso por departamento)**, diseñada para proteger la confidencialidad de la información. El gran reto estuvo en cumplir la "I" (**Independiente**) y la "S" (**Small / Pequeña**) de las reglas INVEST.

- **El problema de fondo:** Al ser un requisito de seguridad transversal, rozaba la lógica de asignación y las bandejas personales de todos los actores.
- **El freno del gate:** El validador dor-invest-gate.py me bloqueaba el avance porque la historia inicialmente contenía términos vagos y dependencias complejas. Me obligó a reescribirla con criterios de aceptación hiperespecíficos para evitar que se convirtiera en una épica gigante e inmanejable.

3. ¿Para qué te serviría un gate de Definition of Ready en tu equipo real?

En mi día a día laboral, el archivo dor-invest-gate.py sería la herramienta ideal para actuar como un **filtro estricto contra la improvisación**. Es sumamente común que en los equipos reales se empujen requerimientos incompletos o ambiguos basados en ideas sueltas.

Tener este gate automatizado nos serviría para:

- **Garantizar la madurez de los datos:** Asegura que nadie empiece a programar un requerimiento si este no cuenta con criterios de aceptación claros o si supera el límite estimado de puntos (MAX_POINTS), protegiendo el tiempo y el enfoque real del equipo de desarrollo.